Facens

Disciplina

Algoritmos e Programação

Conteúdo

Strings

Ao final desta aula o estudante deverá

Ser capaz de utilizar convenientemente strings em Linguagem C

Facens

Strings

Variáveis char

As variáveis alfanuméricas são especificadas por apenas 1 (um) único carácter delimitado por **aspas simples**.

Exemplo:

char letra = ‘a’; // contém o carácter **a** = o código 97 em ASCII

portanto:

printf(“%c”, letra);// saída na tela: a

printf(“%i”, letra); // saída na tela: 97

- **Como as variáveis do tipo char só podem conter um único carácter, se quisermos armazenar 2 ou mais caracteres teremos que fazer uso de uma string.**

Facens

Strings

Definição:

É uma **sequência de caracteres** ou uma “matriz” de caracteres.

Strings Constantes ou Literais - é uma sequência de caracteres delimitada por **aspas duplas**.

Sempre que o compilador encontrar qualquer coisa entre **aspas duplas**, ele reconhece como uma string constante e automaticamente acrescenta o **caracter null (\0)** no final da string

Exemplo:

“a” ⟶ ‘a’ + ‘\0’ ⟶ a\0

a	\0

Facens

Strings

Exemplo:

“a” ⟶ ‘a’ + ‘\0’ ⟶ a\0

a	\0

“casa” ⟶ ‘c’ + ‘a’ + ‘s’ + ‘a’ + ‘\0’ ⟶ casa\0

c	a	s	a	\0

- String é uma sequência de caracteres terminado com o **carácter null**

Facens

Strings - armazenamento

Matriz (vetor) de Caracteres

Para armazenar um string, devemos declarar uma matriz (vetor) do tipo **char** com a quantidade de elementos que queremos mais um, ou seja, 1 para cada carácter e o último para o carácter null (\0).

sintaxe:

char nome_variável[qtde+1];

Exemplo: ”casa”

char str[5];

Facens

Strings - inicialização

Inicialização da Matriz (vetor) de Caracteres

Exemplo: “casa”

```
char str[5] = {‘c’,’a’,’s’,’a’,’\0’};
```

ou

```
char str[5]=“casa”;
```

ou

```
char str[ ]=“casa”;
```

Facens

Strings - leitura

Lendo Strings

Uma vez declarada a string, podemos recebê-la e para isto temos pelo menos 3 funções: scanf, gets e fgets, todas da biblioteca <**stdio.h**>.

- **scanf**

A função scanf( ) é limitada para leitura de strings, pois ela usa qualquer espaço em branco (espaço, carácter de tabulação ou nova linha) para terminar a entrada.

Por exemplo: se você quiser digitar seu nome e sobrenome numa mesma string não conseguirá, pois ao encontrar um espaço em branco, a função encerra e só armazena o nome, desconsiderando o que vier em seguida.

Facens

Strings - leitura

Exemplo:

```
char    nome[80];
printf("Digite seu nome e sobrenome");
scanf("%s", nome);
printf("\n Bom dia "%s", nome);
```

aparecerá na tela:

Digite seu nome e sobrenome **Hamilton Gomes**
Bom dia **Hamilton**

- Note que **não** usamos o operador & precedendo o 2º argumento em scanf, porque o nome de uma matriz é o seu endereço inicial, portanto seria errado utilizar o operador &

~~scanf("%s", &nome);~~

Facens

Strings - leitura

A função scanf( ) é usada para ler uma **mistura** de tipos de dados numa mesma instrução.

Exemplo:

```
char  mat[20];
int   num;
float preço;

printf("Digite o material, seu nº de estoque e seu preço");
scanf("%s  %i  %f", mat, &num, &preço);
```

Facens

Strings - leitura

- **gets -** lê **strings** do teclado

sintaxe:

gets(nome_string);

A função gets é conveniente para a leitura de strings, pois ela lê caracteres até encontrar o caracter de nova linha('\n'), que é gerado quando a tecla ENTER é pressionada pelo usuário. Portanto caracteres brancos (espaços e tabulações) são aceitos como parte da string.

Exemplo:

```
char    nome[80];
printf("Digite seu nome e sobrenome");
gets(nome);
printf("\n Bom dia %s", nome);
```

aparecerá na tela:

Digite seu nome e sobrenome **Hamilton Gomes**

Bom dia **Hamilton Gomes**

Facens

Strings - leitura

fgets - lê **strings** do teclado (entrada padrão) ou de um arquivo, o que for definido.

sintaxe:

fgets(nome_string, num, entrada);

onde:

num – inteiro indica o limite de caracteres a serem lidos, não esquecer do '\0' (null).

entrada – corresponde a entrada padrão do teclado (**stdin**) ou de um arquivo.

A função fgets é conveniente para a leitura de strings, pois ela lê caracteres até encontrar o caracter de nova linha('\n'), que é gerado quando a tecla ENTER é pressionada pelo usuário ou até que o limite especificado (**num-1**) seja atingido, pois a função fgets lê **num-1** caracteres e coloca automaticamente o null (\0)**.** Portanto caracteres brancos (espaços e tabulações) são aceitos como parte da string.

Facens

Strings

Exemplo:

```
char    nome[80];
printf("Digite seu nome e sobrenome");
fgets(nome, 11, stdin);
printf("\n Bom dia %s", nome);
```

aparecerá na tela:

Digite seu nome e sobrenome **Hamilton Gomes**

Bom dia **Hamilton G** // foi especificado apenas 10 caracteres + \0

Facens

Strings

Observação

Quando misturamos no mesmo programa funções como scanf, gets, entre outras, começamos a ter problemas com as entradas, isso acontece porque as funções trabalham de formas diferentes, ou seja, as teclas são armazenadas no buffer do teclado e quando são retiradas, umas retiram o “enter” do teclado e outras não. Então para resolver este problema devemos limpar o buffer após cada scanf, getchar, gets, etc.

- Para limpar o buffer em Windows: fflush(stdin)
- Para limpar o buffer em Linux: __fpurge(stdin)

Facens

Strings - escrita

Imprimindo Strings

As 3 principais funções são: printf, puts e fputs, todas da biblioteca <**stdio.h**>.

- **printf**

Permite exibir mais de um string, ou ainda misturá-los com texto e/ou variáveis numéricas.

Ex.:

```
char    banco[30] = “Banco do Brasil”;
char    nome[80] = “Hamilton Gomes”;
float   saldo = 978.36;
printf(“O saldo de  %s  no  %s  é de  %.2f  reais”, nome, banco, saldo);
```

aparecerá na tela:

O saldo de Hamilton Gomes no Banco do Brasil é de 978.36 reais

Facens

Strings - escrita

- **puts –** imprime 1 única **string** por vez na tela

sintaxe:

puts(nome_string);

A função puts( ) é o complemento de gets( ) e ela pula linha automaticamente após cada string exibida.

Ex.:

```
char    banco[30] = “Banco do Brasil”;
char    nome[80] = “Hamilton Gomes”;
puts(nome);
puts(banco);
```

aparecerá na tela:

Hamilton Gomes
Banco do Brasil

Facens

Strings - escrita

- **fputs –** grava informações no arquivo

sintaxe:

fputs (str, fptr); // armazena um string por vez no arquivo

onde:

str – string a ser armazenado

fptr – corresponde ao ponteiro do arquivo a ser gravado

Facens

Strings

Cuidados!!!

- Os strings têm sua manipulação levemente dificultada. Não é possível, por exemplo, copiar o conteúdo de um string em outro através da seguinte instrução:

~~string1 = string2~~

- Devemos proceder como fazíamos com matriz, ou seja, teremos um **for** varrendo todos os índices.

Ex.:

```
for (i=0; i<10; i++)
    string1[i] = string2[i];
```

- Posições específicas de um string podem receber valores diretamente.

Ex.:

```
string1[0] = 'a';   // a é um char (deve ser colocado entre aspas simples)
```

Facens

Strings

- **Outras Funções**

A biblioteca **<string.h>** da linguagem C, contém uma série de funções para manipular strings.

As principais são:

- **strlen( )** - retorna o tamanho da string. Identifica o tamanho da string até encontrar o caracter null (\0), ou seja, não conta o null.

sintaxe:

var_int = strlen(string);

Exemplo:

```
int tam;
char str[ ] =“casa”;
tam=strlen(str);     //tam = 4 – não contabiliza o \0 - apenas as letras de casa
```

Facens

Strings

- **strcpy(copia,original)** – copia o conteúdo de original para copia

- **memcpy(copia,original,tamanho)** – copia o conteúdo de original para copia até o tamanho especificado

- **strcmp(str1,str2)** – compara (ordem alfabética) str1 com str2, retornando:
  - zero se forem iguais
  - > 0 (positivo, ou seja: 1) se str1 for maior que str2
  - < 0 (negativo, ou seja: -1) se str1 for menor que str2

- **memcmp(str1,str2,tamanho)** – compara (ordem alfabética) str1 com str2 até o número de caracteres especificado em tamanho, retornando:
  - zero se forem iguais
  - > 0 (positivo) se str1 for maior que str2
  - < 0 (negativo) se str1 for menor que str2

Facens

Strings

- **strcat(str1,str2)** – concatena (junta) str1 com str2 ou seja, coloca o conteúdo de str2 no final de str1

Exemplo:

```
char str1[ ]="Ana";
char str2[ ] ="Paula";
strcat(str1,str2);
str1[ ]="AnaPaula";
```

Facens

Resumo

Strings

- String é uma sequência de caracteres terminado com o **caracter null (\0)**
- Para armazenar: vetor tipo **char** com tamanho de caracteres +1 **(\0)**

Strings

- Leitura: scanf(), **gets()**,fgets()
- Escrita: **printf()**, puts(),fputs()
- **Atenção**:

  ~~string1 = string2~~ **for** – elemento à elemento

Facens

Strings

Exemplo1:

– Escreva um programa que receba uma frase e uma letra e a seguir escreva a quantidade de vezes que a letra ocorre na frase.

A	l	u	n	o	s		d	a		F	a	c	e	n	s	\0							

letra a ser procurada: **a** - resposta: 3 ??

Cuidado: linguagem C faz diferença entre letras maiúsculas e minúsculas

Facens

Strings

Exemplo1:

```
#include<stdio.h>
#include<ctype.h> //Para usar a toupper()
#include<string.h> //Para usar a função strlen()
main()
{
    char letra, frase[80];
    int i,tam,cont=0;
    printf("Digite uma frase: ");
    gets(frase);
    fflush(stdin);
    tam=strlen(frase);
    printf("Digite uma letra: ");
    scanf("%c",&letra);
    fflush(stdin);
    for(i=0;i<tam;i++)
      if(toupper(frase[i])==toupper(letra))
         cont++;
    printf("\nA letra %c ocorre %i vezes na frase", letra,cont);
}
```

Strings

Exercício 1:

- Solicite ao usuário que digite uma senha composta por 6 caracteres e na sequência pergunte novamente a senha. Confira se as duas sequências de caracteres são iguais e em caso afirmativo, exiba uma mensagem dizendo que a senha é valida. Caso os três primeiros caracteres sejam iguais, informe que houve algum erro na digitação, caso contrário, a senha será inválida.

Facens

Exercício – nº 2

Faça um programa que, dado um string qualquer, inverta todas as suas letras (no mesmo string).

Para isso iremos precisar da ajuda de uma variável auxiliar (aux)

guarda a letra da 1ª posição 'A' em **aux** e coloca a letra da última posição 's' na 1ª posição

substitui 'A' (**aux**) na última posição

repetir as operações até trocar todas as letras

Exercício – nº 3

Melhore o programa, armazenando a nova sequência em um novo string.

frase

A	l	u	n	o	s		d	a		F	a	c	e	n	s	\0						

invertida

s	n	e	c	a	F		a	d		s	o	n	u	l	A	\0						

Facens

Exercício – nº 4

Melhore o programa, trocando as maiúsculas por minúsculas e vice-versa, quando necessário. Utilizar variável auxiliar.

frase

A	l	u	n	o	s		d	a		F	a	c	e	n	s	\0							

frase

S	n	e	c	a	f		A	d		S	o	n	u	l	a	\0							

Facens

Exercício – nº 5

Melhore o programa, trocando as maiúsculas por minúsculas e vice-versa, quando necessário. Armazenar em um novo string.

frase

A	l	u	n	o	s		d	a		F	a	c	e	n	s	\0							

invertida

S	n	e	c	a	f		A	d		S	o	n	u	l	a	\0							

Facens